

Auditoría de Seguridad: Sesiones, Roles y Control de Acceso Multi-Sucursal

Proyecto: H.A.M.M.E.R. POS/ERP

Fecha: 12 de mayo de 2026

Auditor: Arquitecto de Seguridad Senior (AI)

Alcance: Sesiones, revocación de tokens, RBAC, control de acceso multi-sucursal

1. Resumen Ejecutivo

Se auditó el sistema de autenticación, sesiones, permisos y control de acceso multi-sucursal del proyecto H.A.M.M.E.R. Se encontraron **3 vulnerabilidades P0 (Críticas)**, **4 P1 (Altas)** y **2 P2 (Media)**. Todas las vulnerabilidades P0 fueron corregidas en esta iteración.

2. Análisis del Sistema de Sesiones (Estado Pre-Auditoría)

2.1 Creación de Sesiones

- **Archivo:** `src/modules/auth/service.ts` → `authenticate()`
- **Algoritmo de firma:** HMAC-SHA256 (stateless tokens)
- **Formato:** `base64url(JSON_payload).signature`
- **Payload incluye:** `userId`, `username`, `roleCode`, `branchIds`, `branchMemberships`, `globalRoles`, `primaryBranchId`, `exp`
- **Almacenamiento en DB:** No (tokens stateless, revocación via tabla `RevokedSession`)
- **Evaluación:**  Correcto. HMAC-SHA256 con `timingSafeEqual` para verificación.

2.2 Configuración de Cookies

- **Archivo:** `src/modules/auth/service.ts` → `setSessionCookie()`
- **Flags:**
- `httpOnly: true` 
- `secure: true` (producción) 
- `sameSite: "strict"` 
- `path: "/"` 
- `maxAge:` Configurable via `AUTH_SESSION_TTL_HOURS` 
- **Evaluación:**  Configuración de cookies segura.

2.3 Validación de Sesiones

- **Archivo:** `src/modules/auth/service.ts` → `getCurrentSession()`
- **Verificaciones pre-auditoría:**
-  Valida firma HMAC-SHA256 con `timingSafeEqual`
-  Verifica expiración (`exp`)

-  Consulta tabla `RevokedSession` para tokens revocados individualmente
-  **NO** validaba que el usuario aún existiera en DB
-  **NO** validaba que el usuario estuviera activo (`isActive`)
-  **NO** invalidaba tokens cuando cambiaban roles/contraseña

2.4 TTL (Time To Live)

- **Configurable:** Sí, via `AUTH_SESSION_TTL_HOURS` (default: 12 horas)
- **Refresh automático:** No
- **Límite de sesiones concurrentes:** No (stateless)
- **Evaluación:** ⚠ Aceptable pero sin refresh ni límite concurrente.

2.5 Hashing de Contraseñas

- **Archivo:** `src/modules/auth/password.ts`
- **Algoritmo:** PBKDF2, 210,000 iteraciones, SHA-512, salt random 16 bytes
- **Verificación:** `timingSafeEqual` (timing-safe)
- **Evaluación:**  Excelente. Cumple OWASP 2024.

3. Problemas Encontrados y Correcciones

P0 (Crítico) — BLOQUEADORES corregidos

P0-1: Revocación de Sesiones Incompleta

- **Archivo:** `src/modules/security/token-revocation.ts`
- **Problema:** `revokeAllUserSessions()` solo creaba un registro marcador en `RevokedSession` pero **NO invalidaba tokens existentes**. Los tokens stateless seguían siendo válidos hasta su expiración natural.
- **Riesgo:** Usuarios mantienen acceso total después de cambio de contraseña, desactivación o cambio de roles.
- **Escenario de ataque:** Admin degrada a usuario de MASTER a CASHIER → el token existente sigue teniendo permisos de MASTER por hasta 12 horas.
- **Corrección:**
 1. Agregado campo `sessionVersion` al modelo `User` (default: 0)
 2. `revokeAllUserSessions()` ahora incrementa `sessionVersion`
 3. `getCurrentSession()` valida `sessionVersion` del token vs. DB en cada request
 4. Tokens con `sessionVersion` desactualizado son rechazados inmediatamente

P0-2: Sin Verificación de Usuario Activo en Sesiones

- **Archivo:** `src/modules/auth/service.ts`
- **Problema:** `getCurrentSession()` no verificaba si el usuario seguía existiendo o estaba activo.
- **Riesgo:** Usuarios eliminados o desactivados mantienen acceso con tokens existentes.
- **Corrección:** `getCurrentSession()` ahora consulta `user.isActive` y `user.sessionVersion` en cada request.

P0-3: Falta de Revocación en Cambios Críticos de Usuario

- **Archivo:** `src/modules/users/service.ts`
- **Problema:** `updateUser()`, `upsertMembership()`, `updateMembership()`, `removeMembershipFromUser()` **NO revocaban sesiones** al cambiar roles, contraseña o estado activo.

- **Riesgo:** Cambios administrativos no tenían efecto inmediato en sesiones activas.
- **Corrección:**
- `updateUser()` : Revoca sesiones en cambio de contraseña, desactivación y cambio de `globalRole`
- `upsertMembership()` : Revoca sesiones al asignar/modificar membresía de sucursal
- `updateMembership()` : Revoca sesiones al cambiar rol o estado de membresía
- `removeMembershipFromUser()` : Revoca sesiones al eliminar membresía

P1 (Alto) — Corregidos

P1-1: Validaciones RBAC Manuales e Inconsistentes

- **Archivos afectados:** 8 rutas API
- **Problema:** Validaciones de rol hardcodedas con `session.globalRoles.includes("MASTER")` duplicadas en múltiples rutas.
- **Riesgo:** Inconsistencias (ej: `OWNER` NO incluido en chequeo de `MASTER` en algunas rutas), difícil mantenimiento.
- **Corrección:** Creado módulo `src/modules/security/rbac-helpers.ts` con helpers centralizados (`assertMaster`, `assertOwner`, `assertSystemAdmin`, `assertBranchAccess`, `getAllowedBranchIds`). Reemplazadas todas las validaciones manuales.

P1-2: Falta de Validación de `branchId` en Rutas Sensibles

- **Archivos afectados:** `cash-closure/status`, `branch-config/[branchId]`
- **Problema:** Aceptaban `branchId` de query params sin validar que el usuario tuviera acceso a esa sucursal.
- **Riesgo:** Un usuario de Sucursal A podía consultar datos de Sucursal B modificando el `branchId` en la URL.
- **Corrección:** Agregado `assertBranchAccess(session, branchId)` antes de cada consulta.

P1-3: Change-Password Solo Revocaba Sesión Actual

- **Archivo:** `src/app/api/auth/change-password/route.ts`
- **Problema:** Solo revocaba el token actual con `revokeSessionToken()`, no todos los del usuario.
- **Riesgo:** Si un atacante robó un token, el cambio de contraseña no lo invalidaba.
- **Corrección:** Reemplazado por `revokeAllUserSessions()` que incrementa `sessionVersion`.

P1-4: Falta Validación `OWNER` en `cash-closure/reopen`

- **Archivo:** `src/app/api/cash-closure/reopen/route.ts`
- **Problema:** Usaba `globalRoles.includes("MASTER")` manual sin incluir `OWNER`.
- **Corrección:** Reemplazado con `isMaster()` centralizado + `assertBranchAccess()`.

P2 (Medio) — Documentados

P2-1: Sin Límite de Sesiones Concurrentes

- **Impacto:** Un usuario puede tener sesiones ilimitadas en paralelo.
- **Recomendación futura:** Implementar tabla de sesiones activas con límite configurable.

P2-2: Sin Refresh Token

- **Impacto:** Las sesiones expiran y el usuario debe re-autenticarse.
 - **Recomendación futura:** Implementar refresh tokens con TTL más largo para UX mejorada.
-

4. Jerarquía de Roles Documentada

Jerarquía de Roles Globales (descendente):

1. SYSTEM_ADMIN (Administrador del Sistema)
 - Acceso total al sistema, incluida configuración de plataforma
 - Puede gestionar roles y configuración del sistema
 - Acceso a todas las sucursales
2. OWNER (Dueño del Negocio)
 - Acceso total a nivel operativo y de configuración
 - Puede gestionar configuración de módulos de sucursales
 - Acceso a todas las sucursales
3. MASTER (Administrador Maestro)
 - Acceso total a nivel operativo
 - Gestión de usuarios, catálogo, inventario, ventas
 - Acceso a todas las sucursales
 - NO puede acceder a configuración de sistema (system-admin)
4. BRANCH_ADMIN (Administrador de Sucursal)
 - Dashboard, catálogo, inventario, ventas (lectura)
 - Aprobaciones y reportes
 - Solo sucursal(es) asignada(s)
5. SALES (Vendedor)
 - Dashboard, catálogo (lectura), ventas (CRUD)
 - Solo sucursal(es) asignada(s)
6. CASHIER (Cajero)
 - Dashboard, pagos, sesiones de caja
 - Solo sucursal(es) asignada(s)
7. WAREHOUSE (Almacén)
 - Dashboard, inventario, despacho
 - Solo sucursal(es) asignada(s)

Regla de herencia: isMaster() $\equiv$ MASTER v OWNER v SYSTEM_ADMIN

5. Archivos Modificados

Schema y Migración

Archivo	Cambio
prisma/schema.prisma	Agregado <code>sessionVersion Int @default(0)</code> al modelo <code>User</code>
prisma/migrations/ 20260512_add_session_version_to_users/mi- gration.sql	ALTER TABLE para agregar columna

Módulos de Seguridad (Core)

Archivo	Cambio
<code>src/types/auth.ts</code>	Agregado <code>sessionVersion: number</code> a <code>SessionPayload</code>
<code>src/modules/auth/session.ts</code>	<code>decodeSession()</code> parsea <code>sessionVersion</code> del token
<code>src/modules/auth/service.ts</code>	<code>authenticate()</code> incluye <code>sessionVersion</code> en payload; <code>getCurrentSession()</code> valida <code>sessionVersion</code> , <code>isActive</code> y existencia del usuario contra DB
<code>src/modules/security/token-revocation.ts</code>	<code>revokeAllUserSessions()</code> ahora incrementa <code>sessionVersion</code> del usuario
<code>src/modules/security/rbac-helpers.ts</code>	NUEVO — Helpers centralizados: <code>assertMaster</code> , <code>assertOwner</code> , <code>assertSystemAdmin</code> , <code>assertOwnerOrSystemAdmin</code> , <code>assertBranchAccess</code> , <code>getAllowedBranchIds</code> , <code>canAccessBranch</code>

Servicio de Usuarios

Archivo	Cambio
<code>src/modules/users/service.ts</code>	<code>updateUser()</code> revoca sesiones en cambios de password/roles/activación. <code>upsertMembership()</code> , <code>updateMembership()</code> , <code>removeMembershipFromUser()</code> revocan sesiones.

Rutas API — RBAC Centralizado

Archivo	Antes	Después
<code>src/app/api/auth/change-password/route.ts</code>	<code>revokeSessionToken()</code> solo sesión actual	<code>revokeAllUserSessions()</code> todas las sesiones
<code>src/app/api/ai-insights/anomalies/route.ts</code>	Manual <code>global-Roles.includes("MASTER")</code>	<code>assertMaster(session)</code>
<code>src/app/api/ai-insights/discount-suggestions/route.ts</code>	Manual check	<code>assertMaster(session)</code>
<code>src/app/api/ai-insights/discrepancies/route.ts</code>	Manual check	<code>assertMaster(session)</code>
<code>src/app/api/ai-insights/patterns/route.ts</code>	Manual check	<code>assertMaster(session)</code>
<code>src/app/api/ai-insights/refresh/route.ts</code>	Manual check	<code>assertMaster(session)</code>
<code>src/app/api/cash-closure/route.ts</code>	Manual <code>global-Roles.includes</code>	<code>assertMaster(session)</code>
<code>src/app/api/cash-closure/reports/route.ts</code>	Manual <code>global-Roles.includes</code>	<code>assertMaster(session)</code>
<code>src/app/api/timber/pricing/route.ts</code>	Manual <code>global-Roles.includes("MASTER")</code>	<code>assertMaster(session)</code>
<code>src/app/api/timber/trips/route.ts</code>	Manual check	<code>assertMaster(session)</code>
<code>src/app/api/timber/trips/[id]/route.ts</code>	Manual check (PUT, PATCH)	<code>assertMaster(session)</code>

Rutas API — Control de Acceso a Sucursal

Archivo	Cambio
<code>src/app/api/cash-closure/status/route.ts</code>	Agregado <code>assertBranchAccess(session, branchId)</code>
<code>src/app/api/branch-config/[branchId]/route.ts</code>	Agregado <code>assertBranchAccess(session, branchId)</code>
<code>src/app/api/cash-closure/reopen/route.ts</code>	Reemplazado <code>check manual</code> con <code>assertBranchAccess()</code> + <code>isMaster()</code> centralizado

6. Escenarios de Riesgo Resueltos

Escenario 1: Cambio de Roles (RESUELTO)

- **Situación:** Admin cambia rol de usuario de MASTER a CASHIER
- **Antes:** Token existente con permisos MASTER seguía válido hasta expiración (12h)
- **Después:** `sessionVersion` se incrementa → token rechazado inmediatamente

Escenario 2: Cambio de Contraseña (RESUELTO)

- **Situación:** Usuario cambia contraseña por sospecha de compromiso
- **Antes:** Solo se revocaba la sesión actual; otras sesiones seguían activas
- **Después:** Todas las sesiones del usuario se invalidan vía `revokeAllUserSessions()`

Escenario 3: Desactivación de Usuario (RESUELTO)

- **Situación:** Admin desactiva usuario comprometido
- **Antes:** Token existente permitía acceso hasta expiración
- **Después:** `getCurrentSession()` verifica `isActive` en cada request + `sessionVersion` incrementado

Escenario 4: Acceso a Otra Sucursal (RESUELTO)

- **Situación:** Usuario de Sucursal A modifica `branchId=B` en request
- **Antes:** Algunas rutas no validaban acceso a la sucursal
- **Después:** `assertBranchAccess()` valida acceso antes de toda consulta

Escenario 5: Cambio de Membresía de Sucursal (RESUELTO)

- **Situación:** Admin remueve a usuario de Sucursal B
- **Antes:** Token existente seguía incluyendo Sucursal B en `branchMemberships`
- **Después:** `sessionVersion` incrementado → usuario debe re-autenticarse con nueva membresía

7. Pruebas Recomendadas

Test 1: Revocación en Cambio de Contraseña

1. Login como usuario A, guardar cookie de sesión
2. Cambiar contraseña via `/api/auth/change-password`
3. Intentar usar cookie anterior en otra request
4. **Esperado:** 401 Unauthorized (session invalidated by version mismatch)

Test 2: Revocación en Cambio de Roles

1. Login como usuario con rol MASTER
2. Admin cambia `globalRole` a `null` via `/api/master/users/[id]`
3. Intentar acceder a `/api/master/users` con token anterior
4. **Esperado:** Redirección a `/login` (sesión invalidada)

Test 3: Protección de `branchId`

1. Login como usuario de Sucursal A
2. GET `/api/cash-closure/status?branchId=<id_sucursal_B>`
3. **Esperado:** Error FORBIDDEN_BRANCH

Test 4: Desactivación de Usuario

1. Login como usuario, guardar cookie
2. Admin desactiva usuario (`isActive: false`) via API
3. Intentar cualquier request autenticada
4. **Esperado:** Sesión rechazada (user deactivated)

Test 5: Cambio de Membresía

1. Login como usuario con acceso a Sucursal A y B
2. Admin elimina membresía de Sucursal B
3. Intentar acceder a datos de Sucursal B
4. **Esperado:** Sesión invalidada → re-login con nueva membresía

8. Comandos de Migración

```
# Desarrollo
npx prisma migrate dev --name add_session_version_to_users

# Producción
npx prisma migrate deploy

# Verificar
npx prisma db pull # Debería mostrar sessionVersion en User
```

9. Rutas API con Protección Verificada

Rutas que usan `assertMaster()` centralizado

- `/api/ai-insights/*` (5 rutas)
- `/api/cash-closure` (POST)
- `/api/cash-closure/reports` (GET)
- `/api/timber/pricing` (PUT)
- `/api/timber/trips` (POST)
- `/api/timber/trips/[id]` (PUT, PATCH)
- `/api/master/*` (ya usaban `assertMaster` de `auth/access`)

Rutas con validación de `branchId`

- `/api/cash-closure/status` — `assertBranchAccess`
- `/api/branch-config/[branchId]` — `assertBranchAccess`
- `/api/cash-closure/reopen` — `assertBranchAccess`
- `/api/sales/orders` — `canInBranch()` via policies
- `/api/cashier/*` — `canInBranch()` via policies
- `/api/warehouse/dispatch/*` — `canInBranch()` via policies
- `/api/inventory/*` — `hasBranchAccess()` + `assertBranchAccess`
- `/api/transport/*` — `requireBranchCapability()`

Rutas con scope de sucursal automático

- `/api/reports/*` — `resolveReportBranchScope()` filtra por permisos
-

10. Veredicto

Aspecto	Estado
Firma de tokens	✓ HMAC-SHA256 con timingSafeEqual
Cookies	✓ httpOnly, secure, sameSite=strict
Hashing de contraseñas	✓ PBKDF2, 210k iteraciones, SHA-512
Revocación de sesiones	✓ CORREGIDO — sessionVersion + DB check
Validación de usuario activo	✓ CORREGIDO — check en cada request
RBAC centralizado	✓ CORREGIDO — rbac-helpers.ts
Control multi-sucursal	✓ CORREGIDO — assertBranchAccess en rutas faltantes
Rate limiting	✓ 5 intentos / 15 min por username+IP
CSRF	✓ Ya corregido en auditoría anterior
Límite de sesiones concurrentes	⚠ No implementado (P2)
Refresh tokens	⚠ No implementado (P2)

Estado general: SEGURO para producción (con las correcciones aplicadas)